Lab 08 — Questions

Question 1 [Understanding]

A team builds three images: `api:1.4.2-dev`, `api:1.4.2-staging` and `api:1.4.2-prod`, each with its own `application.yaml` copied inside. The production deployment fails on a setting nobody had tested. Explain the flaw in the method, what should have been done instead, and exactly what you lose by building one image per environment.

Question 2 [Prediction]

The `api:env` image contains `ENV APP_PROFILE=image`. The `api.env` file contains `APP_PROFILE=staging`. What is `APP_PROFILE` in each of the following cases, and why does the order of the options not matter?

```
podman run --env-file api.env api:env
podman run --env-file api.env -e APP_PROFILE=prod api:env
podman run -e APP_PROFILE=prod --env-file api.env api:env
APP_PROFILE=shell podman run -e APP_PROFILE api:env
```

Question 3 [Diagnosis]

```
$ podman run -d --name api -p 18080:8080 -e SERVER_PORT=9090 api:1.0
$ curl -s localhost:18080/actuator/health
$ podman logs api
API demarree sur le port 9090 (profil default)
```

The API runs, the logs look normal, `curl` returns nothing. What is wrong, and what does this example say about the relationship between `EXPOSE`, `-p` and the application's configuration?

Question 4 [Analysis]

A colleague runs `podman run -d --name api api:1.0 --db-password=Secr3t` "because the application accepts the password as an argument". List **four** places where that password is readable, with the command that proves it for each, and rank them from most to least serious.

Question 5 [Analysis]

The same colleague switches to `-e DB_PASSWORD=Secr3t`. Is that better? List the places where the value remains visible, then explain what the `POSTGRES_PASSWORD_FILE` convention concretely changes in `podman inspect`.

Question 6 [Understanding]

Explain how Spring Boot links `SPRING_DATASOURCE_URL` to `spring.datasource.url`, what `SPRING_PROFILES_ACTIVE=prod` does, and in which order Spring Boot settles a value present at the same time in `application.yaml` (inside the JAR), in `/config/application.yaml` (mounted) and in an environment variable.

Question 7 [Diagnosis]

A Dockerfile contains:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
ENV SPRING_PROFILES_ACTIVE=prod
ENV SPRING_DATASOURCE_PASSWORD=Pr0d!
COPY .env /app/.env
COPY target/api.jar /app/api.jar
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Point out everything that is questionable, line by line, and propose the corrected Dockerfile, explaining where each piece of moved information must go.

Question 8 [Prediction]

```
printf 'S3cr3t' | podman secret create db_password -
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine
```

What do `podman inspect db --format '{{.Config.Env}}'`, `podman exec db ls -l /run/secrets/` and `podman secret inspect db_password` show? And what must you add to `podman secret inspect` to read the value back — which proves what?

Question 9 [Analysis]

`podman secret` stores values base64-encoded in your home directory, without encryption. A colleague concludes that "it is pointless, you might as well use `-e`". What does `podman secret` still bring compared with `-e`? And what does a secrets manager (Vault…) bring beyond `podman secret`?

Question 10 [Diagnosis]

```
$ podman build --secret id=token,src=token.txt -t app .      # OK
$ rm token.txt
$ podman build -t app .                                      # also OK?!
```

The Dockerfile contains `RUN --mount=type=secret,id=token,required=true npm ci`. Why does the second build succeed without a secret, and what would it take to see it fail? What is the practical consequence for a token that expires?

Question 11 [Understanding]

Compare `ARG NPM_TOKEN` + `RUN npm ci` with `RUN --mount=type=secret,id=npm_token npm ci`: where does the token end up in each case, and which command checks it on the resulting image? Would the multi-stage build from lab 05 be enough to protect the `ARG`?

Question 12 [Analysis]

A `.env` file with development passwords is committed to Git "because it is only dev". Give three distinct reasons why that is a problem even in development, and the replacement practice (files, `.gitignore`, `.dockerignore`, committed example).

Question 13 [Analysis]

In production, a database secret leaked into a support ticket (someone pasted the output of `podman inspect`). Describe the expected response, step by step, and explain why "delete the

comment from the ticket" is not part of it — then what, in the architecture, makes that response fast or slow.